Wangjie Su

Undergraduate | College of Engineering | Peking University
+86 188 6803 2449 | wsu0605@stu.pku.edu.cn
https://github.com/csxq0605 | https://csxq0605.github.io/

Research Interests

Current focus: Reliable AI agents for scientific simulation and industrial workflows, with an emphasis on domain-tool and knowledge integration, long-horizon memory and context, multi-agent orchestration, failure recovery, and evaluation.

Education

Peking University, B.S. in Theoretical and Applied Mechanics **Sep 2023 – Present**
Peking University, Double Degree in Intelligent Science and Technology **Sep 2024 – Present**

- **GPA**: 3.818/4.0
- **Selected Coursework**: Data Structures and Algorithms (**100**), Programming in AI (**100**), Computer Vision, Probability and Statistics, Numerical Analysis, Generative Modeling, Intelligent Robotics, Trustworthy ML, LLMs and NLG, and Physics Simulation for Computer Graphics.

Research Experience

AI + Fusion Energy Series Project *Undergraduate Researcher* **Mar 2026 – Present**
Xinao Technology Innovation Center, Peking University | PIs: Prof. Yunfeng Liang and Prof. Songfang Huang

- **Research problem:** Studying *long-horizon research autonomy*: preserving executable research state across interruptions under open-ended goals, delayed feedback, and finite resources, while advancing only through independent evidence.
- **Research framework:** Developing orchestration/execution, evidence/verification, memory/evolution, and governance/collaboration beneath a replaceable research loop; posing ablations on persistence, verifiability, self-healing, accumulation, and governability.
- **Progress:** Built the long-running substrate in **Nexgent**.

Combustion Laboratory, Peking University *Undergraduate Researcher* **May 2025 – Present**
PI: Prof. Zheng Chen

- **Background:** Studying fundamental combustion problems in energy conversion and propulsion, including flame dynamics, chemical kinetics, transport, ignition, flame propagation and stabilization, and laminar flame speed.
- **Current work:** Exploring mathematical formulations of multiscale combustion processes from conservation laws and reaction mechanisms, together with machine-learning approaches to predicting key quantities and approximating complex processes.
- **Methodological focus:** Comparing the regimes of mechanistic and data-driven models, with attention to physical consistency, interpretability, data efficiency, and out-of-distribution generalization.

Work Experience

Hangzhou Yuanxi Technology Co., Ltd. *Independent Project Lead* **2026.06 – 2026.09**
Early Technical Team Member | System: ManySelves — A General-Purpose Agent Capability Substrate

- **Capability definition:** Reframed an agent team as a versionable capability specification rather than a fixed application or prompt bundle: Identity contracts define roles, boundaries, I/O, and handoffs; Skill documents encode reusable methods and acceptance criteria; a stable runtime realizes both without domain-specific rewrites.
- **Execution abstraction:** Contributed end to end across the frontend, backend, and runtime; independently designed and implemented the runtime kernel and capability packages, with one Main Agent and task-scoped specialists operating on project-local knowledge, state, and artifacts. Typed state, checkpoints, rollback, resumable decisions, and explicit review boundaries make long-horizon behavior inspectable and recoverable.
- **Research platform:** FastAPI exposes only conversation and task submission while model/tool orchestration, state evolution, and artifact provenance remain server-side. This creates a controlled platform for studying capability transfer, multi-agent coordination, memory, and recovery.

Selected Projects

Nexgent: Long-Running Research Coding Harness *Independent Project* **May 2026 – Present**

- **System definition:** Defined each long-horizon research effort as a persistent **Research Run**, not an extended chat. Versioned contracts unify objectives, budgets, branches, attempts, events, artifacts, verification, and termination, separating Agent execution from external evidence-based acceptance.
- **Execution abstraction:** Implemented SQLite WAL storage, cross-process lease/resume, and a typed DAG for control, data, resource, artifact, recovery, and verification dependencies, with exact-input caching and effect-safe concurrency across shared GUI, TUI, and CLI frontends.
- **Self-healing and evolution:** Built an execute → validate → detect → attribute → recover → rerun causal loop. Strategies are promoted only after independent verification and demoted or disabled after failed reuse; simulator, resource, and irreversible side effects share telemetry and approval. The prototype passed 1,103 offline tests and reproduced a two-attempt, one-recovery loop with strict JSONL validation at 0 errors/0 warnings.

oh-my-opencode (now oh-my-openagent) *Open-Source Contributor* **May 2026 – Present**

- **Research question:** Investigated which agent behaviors should remain invariant across OpenCode, Codex, Pi, and Claude Code rather than being entangled with one Harness. Proposed a downward-only Core–Adapter DAG separating reusable policy from Hook, provider, and UI effects.
- **Semantic abstractions:** Defined agent configuration as a zero-dependency core and model fallback as a session-scoped `idle/armed/exhausted` protocol with injected reachability and logging, turning implicit error handling into explicit, portable, and testable state transitions.
- **System validation:** Preserved existing call sites through re-export shims and tested cleanup, re-arming, chain exhaustion, and session isolation.

TinyTensor: CUDA Deep-Learning Framework *Course Project, Extended* **Fall 2024; Oct 2026**

- **Framework:** Built a PyTorch-style package (tensor, reverse-mode autodiff, nn / functional / fused layers, optimizers) on hand-written C++/CUDA kernels with pybind11 bindings; the whole training step runs without CPU read-back and is replayed as a CUDA Graph.
- **Kernels:** bf16 tensor-core convolution (mma.sync, cp.async pipelines, split-K), grouped convolution, BatchNorm, LayerNorm, single-kernel attention, and fused Conv-BN-Add-ReLU / GEMM-GELU layers; operators, whole-network finite differences, and first training steps are tested against PyTorch.
- **Results:** trained ViT-Tiny, ResNet18, ResNeXt26, and VGG16 on ImageNet-1k 64x64 on one A100 (top-1 56.5% / 56.0% / 62.6% / 57.6%), matching PyTorch epoch by epoch; 2.56x / 1.39x faster per epoch than torch.compile on ViT-Tiny / ResNet18, on par on ResNeXt26, 30% slower on VGG16.

Where Is My Codex (Flow Rescue) *Course Project* **Spring 2026**

- **Physics:** Co-developed a 2D fluid-puzzle game and implemented Position-Based Fluids in Python/Taichi with uniform-grid neighbor search, density constraints, terrain collisions, and GPU/CPU backends.
- **Engineering:** Built five playable levels and a Tiled import pipeline; added headless tests and profiling for digging, collision, channel stability, goal collection, grid construction, and constraint solving.

Honors & Awards

- National Scholarship and Merit Student, Peking University (2025)
- Second Prize, Beijing Division, National College Student Mathematical Modeling Competition (2025)
- Outstanding Learning Award and Third Scholarship, Peking University (2024)
- First Prize, Beijing College Student Mathematics Competition (2024)

Skills

Programming: Python, C/C++, CUDA, MATLAB, LaTeX, CMake, Taichi
Tools & Frameworks: PyTorch, PyQt6, Git, Linux, Scikit-learn, Jupyter, OpenFOAM
Languages: Chinese (native); English (CET-6: 613)